

P2 Documentation and Code Project

You are viewing draft content
created with the use of Claude.AI



Propeller 2 • Application Note P2AN005

Cooperative Multitasking with Spin2 TASK Methods

*Run several jobs in one cog — the TASK methods that retire the
hand-coded coroutine*

July 2026

[Version 1.0.0](#)

What You'll Build

Several cooperative jobs sharing one cog — launched, yielded, halted, resumed, and shut down cleanly, all through Spin2's TASK methods instead of a hand-written PASM coroutine.

One shared idea — a task runs until it voluntarily yields, so several jobs take turns inside a single cog — then a small catalog of recipes you choose among by what you need:

- **R1 — two-task round-robin** — two independent blinkers from one cog
- **R2 — cooperative yield** — a long computation that keeps a second job responsive
- **R3 — halt / resume flow control** — a consumer that pauses and wakes its producer
- **R4 — task dashboard** — a live census of every slot, and a clean shutdown

Applies to: P2 (Propeller 2) silicon • Spin2 / PNut v47 or later • P2 Edge or P2 Eval board;
every recipe runs and reports over DEBUG.

Iron Sheep Productions, LLC

P2 Knowledge Base Project

Abstract

A single P2 cog can run several jobs at once — a blinker, a compute loop, a poller — by letting each job run until it voluntarily hands control to the next. Spin2 makes this a language feature: TASKSPIN starts a task, TASKNEXT yields to the next one, and a small family of companions halts, resumes, checks, and stops them. This note shows how to launch cooperative tasks, share a cog cleanly among them, and coordinate them — through four short, runnable programs. It is the modern replacement for the hand-coded PASM coroutine: the same cooperative concurrency, now written in a handful of readable method calls.

What you'll build: four small programs that share one cog — two independent blinkers, a long computation that stays responsive, a producer a consumer can pause and wake, and a supervisor that reports on every task slot and shuts the workers down cleanly.

Prerequisites & Hardware

You should already be comfortable with:

- Reading and writing Spin2 — methods, DAT/VAR blocks, and `DEBUG()`.
- Launching a method in its own cog with `cogspin` (the cog-level parallel that TASKSPIN mirrors inside a cog).
- The idea that a task needs a **stack** — how big is the subject of the companion note **P2AN006, Sizing Cog & Task Stacks** (a companion P2 Knowledge Base publication). Here we hand each task a comfortable fixed stack and move on.

Version gate. The TASK methods are a Spin2 language feature added in **PNut / Spin2 v47**. Every source file that uses them must open with the `{Spin2_v47}` directive **as its first line** — the compiler will not recognize TASKSPIN, TASKNEXT, or their siblings without it.

Hardware:

- A P2 Edge or P2 Eval board. Every recipe reports over DEBUG, so you can watch each one work with no extra parts. R1 toggles two onboard LEDs; the rest are pure DEBUG.
- Build every program with DEBUG enabled — `pnut_ts -d` — because each uses `DEBUG()` to show its work.

The Idea

Give one cog several jobs, and the jobs must agree on how to share it. The P2's cooperative task model makes that agreement simple: **a task runs until it calls TASKNEXT, and only then does another task get a turn.** No task is ever interrupted against its will. Because the hand-off happens only where *you* wrote TASKNEXT, you always know exactly where a switch can occur — which is what makes cooperative tasks easy to reason about.

TASKSPIN is to a task what COGSPIN is to a cog. COGSPIN launches a method in a *new cog*, running truly in parallel; TASKSPIN launches a method as a *task inside the current cog*, running

cooperatively. Up to 32 tasks can share one cog, each with its own stack and its own program counter, all interleaved at the points where they yield.

The trade is deliberate. Real cogs give you *hardware* parallelism — genuinely simultaneous execution, but only eight cogs exist. Cooperative tasks give you *logical* concurrency — many jobs from one cog, at the cost of each job having to yield in good time. When several jobs are mostly waiting (poll this, blink that, service the other), cooperative tasks let one cog stand in for several, and leave the scarce cogs for work that must run in true parallel.

One discipline makes cooperative tasks safe: **finish what you started before you yield**. If a task is halfway through a shared-resource transaction — a bus write, a multi-field update — and yields, a sibling can interleave and corrupt it. Complete the unit of work, *then* CALL TASKNEXT, and the cooperative model needs no locks at all. (Source: the P2 Knowledge Base cooperative-tasking pattern.)

How It Works (in brief)

The whole family is small. Every method below requires the {Spin2_v47} directive at the top of the file.

Method	Form	What it does
TASKSPIN(id, Method(args), @stack)	statement or expression	Start Method as a task. id is 0–31, or NEWTASK for the next free slot. As an expression it returns the assigned task id, or –1 if all 32 slots are full .
TASKNEXT()	statement	Yield to the next runnable task. Control returns here when this task’s turn comes round again.
TASKHALT(id)	statement	Pause a task; its stack and state are preserved. THISTASK halts the caller.
TASKCONT(id)	statement	Resume a halted task where it left off.
TASKCHK(id)	expression	Status of a slot: 0 = free, 1 = running, 2 = halted.
TASKID()	expression	The caller’s own task id (0–31).
TASKSTOP(id)	statement	Stop a task and free its slot for reuse. THISTASK stops the caller.

Two constants stand in for “which task”: **NEWTASK** (the next free slot, for TASKSPIN) and **THISTASK** (the caller itself, for TASKHALT/TASKSTOP). Both carry the value –1; context tells them apart.

A few behaviors are worth knowing before you build:

- **A task that halts itself yields at once** — TASKHALT(THISTASK) pauses the caller and immediately gives the cog to another task. Something else must later TASKCONT it.
- **If every task is halted**, the cog waits for an interrupt to un-halt one (a PASM interrupt can write the halt bits directly).

- **Returning from a task’s top-level method is an implicit TASKSTOP(THISTASK)** — the slot frees itself. If it was the last task, the cog stops.

There is also a register, **TASKHLT**, holding one halt bit per task for bulk monitoring. R4 reads it. One caution up front: its bits run in **reverse** — bit 0 is task 31, bit 31 is task 0 — so read it as a bitmap, not as task $N = \text{bit } N$.

■ **Hardware — where the task table lives.** Each task’s pointer occupies a cog register in the range \$100–\$11F, filling *downward* from \$11F. That range overlaps the area a large inline-PASM `ORG...END` block uses, so heavy multitasking and heavy inline PASM in the same cog compete for the same registers. For ordinary Spin2 tasks (no big inline-PASM blocks) this never comes up. (Source: Chip Gracey clarification, recorded in the P2 Knowledge Base.)

Choosing a Recipe

Pick by what you’re trying to do:

If you want to...	Use	Key methods
run two or more independent jobs from one cog	R1 — round-robin	TASKSPIN, TASKNEXT
keep a job responsive while another does heavy work	R2 — cooperative yield	TASKNEXT cadence
let one task pause and wake another (flow control, synchronized start)	R3 — halt / resume	TASKHALT, TASKCONT, TASKCHK, THISTASK
monitor or coordinate many tasks, and shut them down cleanly	R4 — task dashboard	TASKCHK, TASKID, TASKHLT, TASKSTOP

Each recipe is a complete, runnable program, and each ends with a ♦ **Verify** that tells you how to confirm it on your own bench.

Recipe R1 — Two-Task Round-Robin

Use it when you have two or more independent, mostly-waiting jobs and don’t want to spend a whole cog on each. The classic case is two blinkers at different rates — the “hello world” of coroutines.

The whole program is one method that blinks a pin at a given period, launched twice. `main` starts a second copy as a task, then falls into the first copy itself:

```
{Spin2_v47}
CON
  _clkfreq = 200_000_000

  LED_A      = 56          ' first blinker's LED (Edge LEDs 56-63)
```

continues on next page →

```

LED_B      = 57                ' second blinker's LED
STACK_LONGS = 64              ' per-task stack (deepest call chain)

DAT
  blinkBStack  LONG 0[STACK_LONGS]

PUB main()
  ' Launch the second blinker as a cooperative sibling task, then
  ' let THIS method become the first blinker. Both share the one cog,
  ' taking turns at tasknext().
  taskspin(NEWTASK, blink(LED_B, 250), @blinkBStack)
  blink(LED_A, 100) ' never returns -> becomes steady-state task

PRI blink(pin, period_ms) | nextEdge
  nextEdge := GETCT()
  repeat
    if GETCT() - nextEdge >= 0 ' time for this LED's next edge?
      nextEdge += period_ms * (clkfreq / 1000)
      pint(pin) ' toggle the LED (complete the "transaction")
      tasknext() ' ...THEN yield, so the sibling blinker runs

```

How this works. `TASKSPIN(NEWTASK, blink(LED_B, 250), @blinkBStack)` starts a second run of `blink` as a task, on its own stack, blinking `LED_B` every 250 ms. `NEWTASK` takes the next free slot. `main` then calls `blink(LED_A, 100)` directly — that call never returns, so `main`'s own task simply *becomes* the 100 ms blinker. Now two tasks exist: whenever one reaches `tasknext()`, the other gets the cog. Each blinker checks a deadline with the standard `GETCT() - nextEdge >= 0` pattern, toggles its LED only when its own period has elapsed, and yields every pass. Because each toggle is complete before the yield, neither blinker ever catches the other mid-update.

★ **Tip — the last method becomes a task for free.** You don't have to launch *every* job with `TASKSPIN`. Launch the siblings, then just *call* the final one; the calling task turns into it. That's why `main` here spawns one task and runs the other inline.

◆ **Verify:** The P2 Edge module's onboard LEDs on pins 56 and 57 are pre-wired — nothing to hook up — so you'll see `LED_A` blink at about 5 Hz and `LED_B` at about 2 Hz, independently, the tell-tale of two tasks truly interleaving. To watch it without LEDs, add `DEBUG(udec(pin))` next to the `pint`; `DEBUG` will show the two pin numbers ticking at their two rates. If **only one** LED blinks, the other task never got the cog — check that both blinkers **CALL** `tasknext()` every pass (a task that never yields starves its siblings). If **nothing** happens, you built without `DEBUG` or without the `{Spin2_v47}` first line.

Recipe R2 — Cooperative Yield in a Long Computation

Use it when one job is compute-heavy and another must stay responsive — a long calculation that mustn't freeze a poll loop or a UI. The lever is *how often the heavy job yields*.

```

{Spin2_v47}
CON
    _clkfreq = 200_000_000

    STACK_LONGS = 96          ' the cruncher's stack
    YIELD_EVERY = 256         ' responsive task runs every N iters

DAT
    crunchStack  LONG 0[STACK_LONGS]

VAR
    LONG checksum ' cruncher publishes its latest result here

PUB main() | heartbeat
    taskspin(NEWTASK, crunch(), @crunchStack) ' the compute-heavy sibling
    heartbeat := 0
    repeat                                     ' THIS task stays responsive
        heartbeat++
        if heartbeat // 5000 == 0
            DEBUG("responsive: heartbeat ", udec(heartbeat), ...
                " latest checksum ", uhex_long(checksum))
            tasknext()

PRI crunch() | pass, i, acc
    pass := 0
    repeat
        acc := 0
        repeat i from 1 to 100_000 ' a deliberately long inner loop
            acc += i
            if i // YIELD_EVERY == 0
                tasknext() ' yield mid-computation -> responsive task runs
        checksum := acc ' publish the result
        pass++
        DEBUG("crunch pass ", udec(pass), ...
            " done, checksum ", uhex_long(checksum))
        tasknext()

```

How this works. The `crunch` task runs a long inner loop, but every `YIELD_EVERY` iterations it calls `tasknext()` — so the responsive `main` task keeps getting turns even though the computation is far from finished. `crunch` finishes a full pass, publishes its result into the shared `checksum`, and yields again. Notice *where* it publishes: `checksum := acc` happens **after** the inner loop, as one atomic long write, never mid-computation — so the responsive task never reads a half-built value.

`YIELD_EVERY` is the throttle. Yield too rarely (a large value) and the responsive task stutters between long compute bursts; yield too often (a small value) and the yield overhead slows the computation. Tune it to the responsiveness you need.

▲ **Pitfall — a shared value that must survive a yield needs care.** `crunch` writes `checksum` only when it holds a finished value, and writes it in one long. If instead you updated a *multi-long* shared structure in pieces and yielded between the pieces, the responsive task could read a torn value. Either finish the update before yielding, or guard it — the same rule as R1’s “complete the transac-

tion first.” (The worked cross-cog version of this is application note **P2AN007, Data Structures with the New Language Facilities**, a companion P2 Knowledge Base publication.)

◆ **Verify:** DEBUG shows the heartbeat line advancing *between* the crunch pass lines — proof the responsive task runs while the computation is in progress. Raise YIELD_EVERY to a large number (say 100_000, i.e. yield once per pass) and you’ll see the heartbeat go quiet during each pass and only catch up between passes — the responsiveness you gave up. That contrast is the recipe.

Recipe R3 — Halt / Resume Flow Control

Use it when one task should *pause* another — a consumer that stops its producer when a buffer fills, or a coordinator that holds tasks at the start line until everything is ready. TASKHALT and TASKCONT are the pause button; TASKCHK reads whether a task is currently halted.

```
{Spin2_v47}
CON
    _clkfreq = 200_000_000

    STACK_LONGS = 64
    BUF_LONGS    = 8 ' power of two -> & mask below is fast modulo
    BUF_MASK     = BUF_LONGS - 1

DAT
    prodStack    LONG 0[STACK_LONGS]

VAR
    LONG buffer[BUF_LONGS]
    LONG head, tail           ' producer owns head, consumer owns tail
    LONG prodId

PUB main() | item, seen
    head := 0
    tail := 0
    seen := 0

    ' --- synchronized start: launch producer HALTED, prep, then release ---
    ' expression form -> task id
    prodId := taskspin(NEWTASK, producer(), @prodStack)
    taskhalt(prodId) ' hold it before it runs a step
    ' (prepare shared state here; producer cannot touch it yet)
    taskcont(prodId) ' now both run

    repeat
        if head <> tail ' something to consume?
            item := buffer[tail]
            tail := (tail + 1) & BUF_MASK
            seen++
            if seen // 8 == 0
                DEBUG("consumed ", udec(item), ...
                    " (producer state ", udec(taskchk(prodId)), ")")
            if taskchk(prodId) == 2 ' 2 = halted: space now, wake it
```

continues on next page →

```

        taskcont(prodId)
    tasknext()

PRI producer() | n
    n := 0
    repeat
        if ((head + 1) & BUF_MASK) == tail           ' ring full?
            ' halt SELF (auto-yields); consumer resumes me
            taskhalt(thistask)
        else
            buffer[head] := n++
            head := (head + 1) & BUF_MASK
    tasknext()

```

How this works. `main` captures the producer’s task id from TASKSPIN’s *expression* form, then immediately `taskhalt(prodId)` — the **synchronized start**: the producer is created but held before it runs a single step, so `main` can set up shared state with no risk of the producer touching it. `taskcont(prodId)` releases it.

From then on it is flow control. The producer fills a small power-of-two ring. When the ring is full it calls `taskhalt(thistask)` — halting *itself*, which also yields — so it stops producing and burns no cycles spinning. The consumer drains the ring, and each pass checks `taskchk(prodId) == 2`: if the producer is halted (state 2) and there is now space, it calls `taskcont` to wake it. The producer never has to poll for space; it sleeps until the consumer needs more.

★ **Tip — TASKCHK before TASKCONT.** Waking a task that isn’t halted is harmless, but testing `taskchk(id) == 2` first makes the intent explicit and lets you log the state transition, as the DEBUG line here does.

◆ **Verify:** DEBUG prints a consumed line every eight items, each tagged with the producer’s current state (1 = running, 2 = halted). Under a fast consumer you’ll mostly see state 1; slow the consumer down (add a `waitms(5)` in the loop) and you’ll watch the producer flip to state 2 as the ring fills and back to 1 as it drains — the flow control working. If items **stop** appearing, the producer halted and was never resumed: confirm the consumer’s `taskcont` branch runs (a `<>` vs `==` slip on the state TEST will do it).

Recipe R4 — Task Coordination & Status Dashboard

Use it when you’re running many tasks and need to see them — how many are alive, which are halted — and bring them down cleanly. This recipe takes a live census with TASKCHK, reads the TASKHLT bitmap, and shuts the workers down with a flag each honors via TASKSTOP(THISTASK).

```

{Spin2_v47}
CON
    _clkfreq = 200_000_000

    STACK_LONGS = 64

```

```

N_WORKERS    = 3

DAT
  workerStacks  LONG 0[STACK_LONGS * N_WORKERS]

VAR
  LONG shutdown

PUB main() | i, running, halted, id
  shutdown := 0
  repeat i from 0 to N_WORKERS - 1
    taskspin(NEWTASK, worker(), @workerStacks[i * STACK_LONGS])

  ' --- supervisor: census the slots a few times, then order shutdown ---
  repeat 5
    running := 0
    halted  := 0
    repeat id from 0 to 31
      case taskchk(id) ' 0=free, 1=running, 2=halted
        1: running++
        2: halted++
    DEBUG("census: running=", udec(running), ...
          " halted=", udec(halted), " TASKHLT=", ubin_long(TASKHLT))
    yield_a_while()

  shutdown := 1 ' workers see this and self-terminate
  yield_a_while()
  DEBUG("after shutdown: workers still running = ", udec(worker_census()))

PRI worker() | announced
  announced := 0
  repeat
    if NOT announced
      DEBUG("worker up as task ", udec(taskid())) ' self-identify once
      announced := 1
    if shutdown
      taskstop(thistask) ' free my slot; never returns
      tasknext()

PRI worker_census() : running | id
  ' count only the worker slots (ids above the main task) still running
  repeat id from 1 to 31
    if taskchk(id) == 1
      running++

PRI yield_a_while() | k
  repeat k from 0 to 199
    tasknext()

```

How this works. `main` launches three workers into one contiguous stack block, giving each its own `STACK_LONGS` slice via `@workerStacks[i * STACK_LONGS]`. Each worker announces its own id with `TASKID()` the first time it runs, then simply yields until it sees the shutdown flag — at which point `taskstop(thistask)` frees its slot and the method never returns.

The supervisor is the dashboard. Each round it walks all 32 slots with `TASKCHK`, tallying running (1) and halted (2), and prints the `TASKHLT` register as a raw bitmap. (The census counts the supervisor's own slot as running too — it *is* a task — so with three workers up you'll see running = 4.) After a few rounds it sets shutdown, gives the workers time to notice, and re-counts *only* the worker slots to confirm they've stopped.

▲ **Pitfall — TASKHLT bits are reversed.** The register is handy for a one-glance bitmap, but bit 0 is task 31 and bit 31 is task 0. Don't index it as `task N = bit N`; if you need a specific task's state, use `TASKCHK(N)` (as the census does) rather than picking a bit out of `TASKHLT`.

◆ **Verify:** The census lines show `running=4` (three workers plus the supervisor) while the workers are up, then `worker census = 0` after shutdown — the workers freed their slots. Each worker also prints its assigned id once at startup; you'll see three distinct ids. If the post-shutdown count is **not** zero, a worker isn't reaching its `taskstop` — check that shutdown is a shared VAR (not a local) so every worker can see it.

Adapt It / Going Further

The four recipes combine into real structure. The natural next step is **many cadences on one shared resource**: several tasks in one cog, each servicing a device at its own rate over a bus that a single cog must own. Servos at 50 Hz, an IMU at 100 Hz, a battery gauge at 1 Hz — all on one I²C bus, all cooperative tasks in the bus's owning cog. Each task does its whole bus transaction, then yields; because the yield never lands mid-transaction, the bus stays coherent with no lock. This is exactly the cooperative model paying off: you separate the cadences *without* splitting the bus across cogs (which would need a hardware mutex the P2 doesn't have). The P2 Knowledge Base cooperative-tasking pattern works this example end to end.

Two knobs generalize everything here:

- **Yield placement** decides responsiveness and coherence. Yield after each complete unit of work; never inside one. R2's `YIELD_EVERY` is this knob made explicit.
- **Stack size** decides whether a task survives at all. Every task here uses a comfortable fixed stack; a task whose real `CALL` depth exceeds it will silently corrupt memory. Sizing them properly is a worked recipe of its own — see **P2AN006, Sizing Cog & Task Stacks**.

Pitfalls & Notes

▲ **Pitfall — the version directive is mandatory and must be first.** Without `{Spin2_v47}` as the file's first line, the compiler doesn't know `TASKSPIN`, `TASKNEXT`, or any of the family, and you'll get "expected an instruction" errors on lines that look perfectly correct.

▲ **Pitfall — a task that never yields starves every other task.** Cooperative means voluntary. A loop with no `tasknext()` on some path will hold the cog forever the moment it takes that path. Make sure every loop in every task reaches a yield.

▲ **Pitfall — don’t yield mid-transaction.** If a task shares a bus or a multi-field structure, finish the transaction before `tasknext()`. Yield in the middle and a sibling can interleave and corrupt it. This is the one rule that keeps cooperative tasks lock-free.

▲ **Pitfall — TASKHLT bits are reversed** (bit 0 = task 31). Use it for bulk bitmaps; use `TASKCHK(id)` when you want a specific task’s state.

★ **Tip — the last job runs inline.** Launch N–1 jobs with `TASKSPIN` and simply *call* the last; the calling task becomes it. No stack is allocated for the inline one — it uses the cog’s existing stack.

★ **Tip — self-halt and self-stop are clean idioms.** `TASKHALT(THISTASK)` pauses the caller and yields in one step (wait to be resumed); `TASKSTOP(THISTASK)` — or simply returning from the task’s top method — frees the slot.

■ **Hardware — tasks share the cog’s registers.** All tasks in a cog share its hub/register state; only the stack and program counter are per-task. That’s what makes sharing data between them trivial (just use a `VAR`) and what makes the “finish before you yield” rule necessary.

Conclusion

Cooperative multitasking turns one cog into several logical jobs, and Spin2’s TASK methods make it a matter of a few readable calls: `TASKSPIN` to launch, `TASKNEXT` to yield, and `TASKHALT` / `TASKCONT` / `TASKCHK` / `TASKSTOP` to coordinate. The model’s one discipline — run until you choose to yield, and finish your work before you do — is also its whole safety story: because switches happen only where you wrote them, cooperative tasks stay easy to reason about and need no locks. Reach for them when several mostly-waiting jobs would each waste a whole cog, and keep the real cogs for work that must run in true parallel. When you launch a task, size its stack deliberately — the companion note **P2AN006** shows how.

Resources

- **Example library** — every recipe in this note as a standalone, `pnut_ts`-compiled program: `two-task-round-robin.spin2` (R1), `cooperative-yield.spin2` (R2), `halt-resume-flow.spin2` (R3), `task-dashboard.spin2` (R4). Published as `P2AN005-src-<date>.zip` beside this note.
- **P2AN006 — Sizing Cog & Task Stacks** (a companion P2 Knowledge Base publication) — how big to make the stack each task needs; the companion to this note.
- **P2AN007 — Data Structures with the New Language Facilities** (a companion P2 Knowledge Base publication) — the worked cross-cog version of “share a value safely.”

References

1. *Spin2 Language Documentation*, v47 and later (Chip Gracey, Parallax Inc.) — the TASK method family (`TASKSPIN`, `TASKNEXT`, `TASKHALT`, `TASKCONT`, `TASKCHK`, `TASKID`, `TASKSTOP`), the `NEWTASK` / `THISTASK` constants, and the `TASKHLT` register.

2. P2 Knowledge Base cooperative-tasking pattern — the “finish the transaction before you yield” discipline and the many-cadences-on-one-bus worked example.

Revision History

- **v0.1.0** (July 2026) — initial draft for review. Four runnable recipes (round-robin, cooperative yield, halt/resume flow control, task dashboard), each `pnut_ts -d`-compiled; scheduling and runtime behaviors described from the Spin2 v47+ documentation, hardware confirmation pending.

Copyright & License

Copyright © 2026 Iron Sheep Productions, LLC and Parallax Inc. Licensed under the Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License (CC BY-NC-ND 4.0). “Propeller” and “Parallax” are trademarks of Parallax Inc.

Acknowledgments

Parallax Inc. and Chip Gracey — for the Propeller 2 silicon and the Spin2 language, including the cooperative-task methods this note applies.